

Inheritance

클래스가 다른 클래스를 물려받다 — superclass <, 메서드 상속, `super` 호출

jlox · Java tree-walking interpreter

이 장에서 만드는 것

상속의 세 가지: 물려받기 · 오버라이드 · super 호출

```
class Doughnut {  
  cook() { print "Fry until golden brown."; }  
}  
  
class BostonCream < Doughnut {      // '<' 로 상속  
  cook() {  
    super.cook();                    // 부모 메서드 호출  
    print "Pipe full of custard and coat with chocolate.";  
  }  
}  
  
BostonCream().cook();  
// Fry until golden brown.  
// Pipe full of custard and coat with chocolate.
```

필요한 것

- superclass 선언 (<)
- 메서드 상속 (찾기 위임)

또다시 전 단계를 가로지른다

12장 클래스 위에 얹게 한 겹 — 새 런타임 타입은 없다



새 AST 노드 1개 — `Expr.Super · Stmt.Class`엔 `superclass` 필드(`Expr.Variable`)가 추가된다.

새 런타임 타입 0개 — `LoxClass`에 `superclass` 필드 하나만 더한다. 인스턴스/필드 구조는 그대로.

추가되는 문법

classDecl에 선택적 superclass 절, primary에 super

```
// 선언 - 클래스 이름 뒤에 선택적 "<" 부모이름"
classDecl → "class" IDENTIFIER ( "<" IDENTIFIER )?
           "{" function* "}" ;

// 기본식 - super.method 형태로만 등장 (단독 super 불가)
primary   → ... | "super" "." IDENTIFIER ;
```

왜 < 인가?

상속 키워드(extends 등) 없이 기호 하나로. 스캐너는 이미 <를 알고 있어 토큰 추가 0개.

왜 super 단독 금지?

super 자체는 객체가 아니다 — 항상 super.method로 부모의 메서드를 지목할 때만 의미가 있다.

① superclass — 파싱

```
// Parser.classDeclaration()
private Stmt classDeclaration() {
    Token name = consume(IDENTIFIER, "Expect class name.");

    Expr.Variable superclass = null;
    if (match(LESS)) {                // '<' 가 있으면
        consume(IDENTIFIER, "Expect superclass name.");
        superclass = new Expr.Variable(previous());    // 변수 참조로
    }

    consume(LEFT_BRACE, "Expect '{' before class body.");
    ... // 메서드 파싱은 12장 그대로
    return new Stmt.Class(name, superclass, methods);    // 필드 1개 추가
}
```

부모 이름을 `Expr.Variable`로 감싸는 이유 — `superclass`는 런타임에 변수처럼 평가해서 실제 `LoxClass` 객체를 얻어야 하기 때문. `resolve` 평가를 변수 메커니즘에 그대로 엮는다.

① superclass — 실행

```
// Interpreter.visitClassStmt() (앞부분)
Object superclass = null;
if (stmt.superclass != null) {
    superclass = evaluate(stmt.superclass);           // 변수 평가
    if (!(superclass instanceof LoxClass)) {         // 런타임 검사
        throw new RuntimeError(stmt.superclass.name,
                                "Superclass must be a class.");
    }
}

environment.define(stmt.name.lexeme, null);
...
LoxClass klass = new LoxClass(stmt.name.lexeme,
                               (LoxClass)superclass, methods); // 부모를 품고 생성
```

왜 런타임 검사? `class Eclair < epicScone {}` 처럼 부모 자리에 **클래스가 아닌 값**이 올 수 있다. 이걸 정적으로 알 수 없어 런타임 에러로 잡는다.

② 메서드 상속 — findMethod 위임

상속의 본질은 의외로 단순하다 — 없으면 부모에게 물어본다

```
// LoxClass.java
class LoxClass implements LoxCallable {
    final String name;
    final LoxClass superclass;           // ← 추가된 필드
    private final Map<String, LoxFunction> methods;

    LoxFunction findMethod(String name) {
        if (methods.containsKey(name)) { // ① 내 메서드 먼저
            return methods.get(name);
        }
        if (superclass != null) {       // ② 없으면 부모에게
            return superclass.findMethod(name); // 재귀적으로 거슬러 올라감
        }
        return null;                    // ③ 끝까지 없으면 null
    }
}
```

12장의 get·init 호출이 모두 이미 findMethod를 거친다 → 이 한 군데만 고치면 **오버라이드도 공짜**. 자식에 있으면 자식 것이 먼저 잡힌다.

③ super — 왜 어려운가

`super.cook()`은 "나를 정의한 클래스의 부모"에서 찾아야 한다

```
class A          { method(){ "A"; } }
class B < A      {
  method(){ "B"; }
  test(){ super.method(); } // → "A"
}
class C < B      { }

C().test();      // 무엇이 출력될까?
```

함정: `this`는 C 인스턴스다. 만약 `super`를 "`this`의 부모"로 풀면 C의 부모 B → B.method → "B" (오답!)

정답은 "A" — `super`는 `test`가 선언된 클래스(B)의 부모(A)에서 찾아야 한다. 인스턴스의 타입이 아니라 메서드가 쓰인 위치가 기준.

즉 `super`는 정적으로(렉시컬하게) 결정된다 — 그래서 `this` 환경 바로 바깥에 `super` 환경을 끼워 넣는다.

③ super — 환경 한 겹을 끼운다

```
// Interpreter.visitClassStmt()
environment.define(stmt.name.lexeme, null);

if (stmt.superclass != null) {
  environment = new Environment(environment);
  environment.define("super", superclass); // super 한 칸
}

... // 이 환경에서 메서드들을 LoFunction으로 생성
// → 각 메서드의 closure가 super를 본다

if (superclass != null) {
  environment = environment.enclosing; // 원복
}
```

메서드 본문이 보는 환경 체인:

this 환경 this → instance

↓ enclosing

super 환경 super → 부모 클래스

↓ enclosing

클래스 정의 환경

③ super — 파싱과 평가

파싱: super.method

```
// Parser.primary()
if (match(SUPER)) {
    Token keyword = previous();
    consume(DOT,
        "Expect '.' after 'super'.");
    Token method = consume(IDENTIFIER,
        "Expect superclass method name.");
    return new Expr.Super(keyword, method);
}
```

평가: 두 칸을 꺼낸다

```
// Interpreter.visitSuperExpr()
int distance = locals.get(expr);
LoxClass superclass = (LoxClass)
    environment.getAt(distance, "super");
LoxInstance object = (LoxInstance)
    environment.getAt(distance - 1, "this");

LoxFunction method =
    superclass.findMethod(expr.method.lexeme);
if (method == null)
    throw new RuntimeError(expr.method,
```

③ super — Resolver가 스코프를 씌운다

```
// Resolver.visitClassStmt()
if (stmt.superclass != null &&
    stmt.name.lexeme.equals(stmt.superclass.name.lexeme)) {
    Lox.error(stmt.superclass.name, "A class can't inherit from itself."); // 정적 검사
}

if (stmt.superclass != null) {
    currentClass = ClassType.SUBCLASS;           // "지금 자식 클래스 안"
    resolve(stmt.superclass);                     // 부모 이름을 변수로 resolve
    beginScope();
    scopes.peek().put("super", true);            // ← super 스코프 한 칸
}

beginScope(); scopes.peek().put("this", true);   // 그 안에 this 스코프
... // resolveFunction(method, ...)
endScope();                                     // this 닫기
if (stmt.superclass != null) endScope();        // super 닫기
```

스코프를 **super** → **this** 순서로 쌓아, 인터프리터의 환경 체인과 **거리(distance)**가 정확히 일치하게 만든다.

Resolver가 런타임 전에 잡아주는 것

자기 자신 상속 금지

```
// class Oops < Oops {}  
if (superclass 이름  
    == 클래스 이름)  
    Lox.error(...,  
        "A class can't  
        inherit from itself.");
```

super 사용 위치 검사

```
// visitSuperExpr  
if (currentClass==NONE)  
    Lox.error("Can't use 'super'  
        outside of a class.");  
else if (currentClass  
        != SUBCLASS)  
    Lox.error("Can't use 'super'  
        in a class with  
        no superclass.");
```

한눈에 보는 super.cook() 한 줄

BostonCream < Doughnut 에서 BostonCream().cook() 호출 시



오버라이드 — BostonCream.cook이 먼저 잡혀 자식 버전 실행.

super.cook — 부모 Doughnut.cook을 같은 인스턴스에 bind해 실행 → 둘 다 찍힌다.

정리

추가된 것

새 AST / 필드

Expr.Super

Stmt.Class.superclass

런타임

- `LoxClass.superclass` 필드 1개
- `findMethod` → 부모로 재귀 위임
- `super` 환경 한 겹 (`this` 바깥)

핵심 통찰

- 상속 = `findMethod` 부모 위임이 전부
- 오버라이드는 자식 먼저 조회로 공짜
- 부모는 변수처럼 평가 (`Expr.Variable`)
- `super`는 선언 위치의 부모 (정적)
- `super` 환경 + `distance-1`로 `this` 회수
- 자기상속·잘못된 `super`는 **Resolver**가 차단